Code Explainer Manual

Travel Planner Agent

A module-by-module guide to the backend, frontend, and test code in the repository.

Generated on 2026-07-04 14:42 from
/Users/naveen.kumar.p/Desktop/pprojects/travel-planner-agent.

Scope note: this manual explains repo-owned code modules under backend source, backend tests, backend scripts, and frontend source. Generated assets, caches, third-party packages, and docs-only files are intentionally excluded.

How To Use This Manual

This manual is organized around code packages rather than user stories. Read it when you need to understand where logic lives, which modules own specific responsibilities, and how backend and frontend pieces fit together.

The backend should be read from the outside inward: entrypoint, API, services, orchestration, providers, persistence, and tests. The frontend should be read from the outside inward as well: router, page modules, shared components, layout, and client integration layer.

A good reading rhythm is package summary first, then the file-level entries in that package, then the corresponding tests. That gives you both architecture intent and executable proof of behavior.

Manual Outline

- Backend package map
- Frontend package map
- Test-suite explanations
- Module-by-module appendix

Code Inventory Snapshot

```
backend/src: 101 modules
backend/tests: 16 modules
backend/scripts: 1 module
frontend/src: 46 modules
total code modules documented: 164
```

Backend Overview

The backend is a layered Python application built around FastAPI, LangGraph, a custom coordinator-led runtime, Redis-backed asynchronous execution, and Postgres persistence. The important design decision is that orchestration, provider access, storage, and API contracts are kept in distinct packages rather than being mixed into one file or one service.

Two orchestration models exist side by side. The older sequential graph is still present because it provides a simple, understandable workflow baseline. The newer coordinator runtime adds explicit roles, task ledgers, selective parallelism, and more agent-like behavior. Both are worth understanding because the repository's current design has evolved through that transition.

The backend tests are not an afterthought. They document intended behavior for configuration, providers, runtime coordination, observability, review flows, and resilience. Use them as a second form of architecture documentation.

Frontend Overview

The frontend is a React and TypeScript application organized around route-level pages and a central planner client. Each major screen corresponds to a backend artifact family or user workflow stage: new trip, clarification, research, itinerary, budget, review, dashboard, and auth.

The most important frontend file is `frontend/src/lib/planner.ts` because it is the contract bridge between browser pages and backend APIs. It owns fetch helpers, runtime-polling helpers, local storage state, and shared frontend types. Route modules then consume that library rather than inventing their own transport logic.

Reusable UI modules under `frontend/src/components` keep the pages from collapsing into giant single-file implementations. CSS modules remain near the components or pages they style, which keeps visual concerns local and discoverable.

Package Explanations

backend/scripts

Backend operational utility scripts used for probes or load checks.

Modules in this package: 1

backend/src

Backend source root. This is the main Python application tree. It holds the HTTP layer, orchestration runtime, services, provider clients, repositories, domain contracts, and worker entrypoints.

Modules in this package: 3

backend/src/agents/travel_planner

Planner core. This is where trip-planning state, specialist nodes, prompts, routing helpers, governance checks, and the original sequential graph are implemented.

Modules in this package: 11

backend/src/agents/travel_planner/multi_agent

Coordinator-led multi-agent runtime. These modules define agent roles, coordination state, delegation rules, runtime scheduling, and adapter logic that reuses the older specialist implementations.

Modules in this package: 6

backend/src/agents/travel_planner/tooling

Planner tool abstraction layer. These files keep provider-facing capabilities organized as planner-owned tools rather than scattering raw external API logic into every specialist.

Modules in this package: 10

backend/src/api

Backend HTTP boundary. Modules here accept requests, enforce auth, call services or use cases, and return typed responses.

Modules in this package: 1

backend/src/application/admin

Application-layer contracts and mappers. These modules shape internal state into API-safe schemas and isolate transport-facing representations from storage and domain models.

Modules in this package: 1

backend/src/application/audit

Application-layer contracts and mappers. These modules shape internal state into API-safe schemas and isolate transport-facing representations from storage and domain models.

Modules in this package: 2

backend/src/application/auth

Application-layer contracts and mappers. These modules shape internal state into API-safe schemas and isolate transport-facing representations from storage and domain models.

Modules in this package: 1

backend/src/application/jobs

Application-layer contracts and mappers. These modules shape internal state into API-safe schemas and isolate transport-facing representations from storage and domain models.

Modules in this package: 2

backend/src/application/observability

Application-layer contracts and mappers. These modules shape internal state into API-safe schemas and isolate transport-facing representations from storage and domain models.

Modules in this package: 1

backend/src/application/trips

Application-layer contracts and mappers. These modules shape internal state into API-safe schemas and isolate transport-facing representations from storage and domain models.

Modules in this package: 3

backend/src/application/workflows

Application-layer contracts and mappers. These modules shape internal state into API-safe schemas and isolate transport-facing representations from storage and domain models.

Modules in this package: 4

backend/src/core

Cross-cutting backend infrastructure. Settings, logging, request context, redaction, response shaping, and security live here.

Modules in this package: 6

backend/src/db

SQLAlchemy database setup. These modules create the engine and sessions and define the ORM tables.

Modules in this package: 4

backend/src/domain/audit

Domain model layer. Business concepts such as trips, jobs, workflows, and audit records are defined here without tying them directly to FastAPI or SQLAlchemy routes.

Modules in this package: 2

backend/src/domain/jobs

Domain model layer. Business concepts such as trips, jobs, workflows, and audit records are defined here without tying them directly to FastAPI or SQLAlchemy routes.

Modules in this package: 2

backend/src/domain/trips

Domain model layer. Business concepts such as trips, jobs, workflows, and audit records are defined here without tying them directly to FastAPI or SQLAlchemy routes.

Modules in this package: 5

backend/src/domain/workflows

Domain model layer. Business concepts such as trips, jobs, workflows, and audit records are defined here without tying them directly to FastAPI or SQLAlchemy routes.

Modules in this package: 4

backend/src/evals

Evaluation helpers used to compare planner behavior against expected outcomes or golden cases.

Modules in this package: 5

backend/src/messaging

Async execution plumbing. These modules connect the application to Redis-backed queue behavior and background jobs.

Modules in this package: 2

backend/src/persistence/memory

In-memory repository implementations for tests or lightweight flows.

Modules in this package: 4

backend/src/persistence/postgres

Postgres-backed repository implementations used by the real backend runtime.

Modules in this package: 7

backend/src/providers

External provider client layer. These modules know how to talk to OpenAI, Tavily, SerpApi, WeatherAPI, and Aviationstack.

Modules in this package: 6

backend/src/services

Service layer. This is where orchestration-aware business operations are coordinated across repositories, runtimes, and policies.

Modules in this package: 8

backend/src/workers

Worker-process entrypoints that run queued workflow jobs outside the web server.

Modules in this package: 1

backend/tests

Backend automated tests. These modules describe the intended behavior of configuration, orchestration, providers, observability, security, and resilience.

Modules in this package: 16

frontend/src

Frontend source root. It contains the application router, global CSS, route-level pages, reusable components, shared layouts, and the client-side planner integration layer.

Modules in this package: 4

frontend/src/app

Route-level React pages. Each module corresponds to a major user-facing screen such as new trip, clarification, itinerary, budget, review, or research.

Modules in this package: 1

frontend/src/app/budget

Route-level React pages. Each module corresponds to a major user-facing screen such as new trip, clarification, itinerary, budget, review, or research.

Modules in this package: 1

frontend/src/app/clarification

Route-level React pages. Each module corresponds to a major user-facing screen such as new trip, clarification, itinerary, budget, review, or research.

Modules in this package: 1

frontend/src/app/dashboard

Route-level React pages. Each module corresponds to a major user-facing screen such as new trip, clarification, itinerary, budget, review, or research.

Modules in this package: 2

frontend/src/app/itinerary

Route-level React pages. Each module corresponds to a major user-facing screen such as new trip, clarification, itinerary, budget, review, or research.

Modules in this package: 1

frontend/src/app/login

Authentication screens. These files implement user login and signup flows in the browser.

Modules in this package: 3

frontend/src/app/new-trip

Route-level React pages. Each module corresponds to a major user-facing screen such as new trip, clarification, itinerary, budget, review, or research.

Modules in this package: 1

frontend/src/app/research

Route-level React pages. Each module corresponds to a major user-facing screen such as new trip, clarification, itinerary, budget, review, or research.

Modules in this package: 1

frontend/src/app/review

Route-level React pages. Each module corresponds to a major user-facing screen such as new trip, clarification, itinerary, budget, review, or research.

Modules in this package: 1

frontend/src/components

Reusable UI building blocks shared across multiple pages.

Modules in this package: 26

frontend/src/layouts

Shared application layout modules that provide page chrome and structural consistency.

Modules in this package: 2

frontend/src/lib

Frontend integration library. This is where API calls, local storage helpers, and shared frontend types live.

Modules in this package: 2

Module Appendix

backend/scripts

Backend operational utility scripts used for probes or load checks.

`backend/scripts/load_test.py`

Backend load-test utility for quick endpoint pressure checks.

backend/src

Backend source root. This is the main Python application tree. It holds the HTTP layer, orchestration runtime, services, provider clients, repositories, domain contracts, and worker entrypoints.

`backend/src/__init__.py`

Code module `backend/src/__init__.py`. It belongs to the repository-owned implementation surface and contributes to runtime behavior, UI rendering, or automated verification.

`backend/src/bootstrap.py`

Dependency composition root. This module wires settings, repositories, services, runtimes, provider clients, queue support, and dependency-injection helpers.

`backend/src/main.py`

FastAPI process entrypoint. It assembles the web application and mounts the API router.

backend/src/agents/travel_planner

Planner core. This is where trip-planning state, specialist nodes, prompts, routing helpers, governance checks, and the original sequential graph are implemented.

`backend/src/agents/travel_planner/contracts.py`

Typed input/output contract helpers for specialist nodes.

`backend/src/agents/travel_planner/governance.py`

Governance rules for confidence checks, early routing, and policy-oriented review signals.

`backend/src/agents/travel_planner/graph.py`

Original sequential LangGraph workflow. It registers the step order for clarification, research, itinerary, stay, transport, food, budget, safety, review, and governance.

`backend/src/agents/travel_planner/nodes.py`

Primary specialist implementation module. This is where most planner artifacts are actually produced.

`backend/src/agents/travel_planner/prompts.py`

Prompt templates used by planner specialists.

`backend/src/agents/travel_planner/research_clients.py`

Helper integrations for travel research tasks.

`backend/src/agents/travel_planner/research_prompts.py`

Prompt templates focused on research-oriented planner behavior.

`backend/src/agents/travel_planner/routing.py`

Small routing helper module that decides how the workflow moves after clarification or other checkpoints.

backend/src/agents/travel_planner/schemas.py

Core planner contract file. It defines trip requests, clarification models, and structured output artifacts used across backend and frontend.

backend/src/agents/travel_planner/state.py

Planner runtime state definitions shared by orchestration and specialists.

backend/src/agents/travel_planner/tools.py

Planner-level tool entry module tying tool abstractions into the travel planner subsystem.

backend/src/agents/travel_planner/multi_agent

Coordinator-led multi-agent runtime. These modules define agent roles, coordination state, delegation rules, runtime scheduling, and adapter logic that reuses the older specialist implementations.

backend/src/agents/travel_planner/multi_agent/__init__.py

Code module `backend/src/agents/travel_planner/multi_agent/__init__.py`. It belongs to the repository-owned implementation surface and contributes to runtime behavior, UI rendering, or automated verification.

backend/src/agents/travel_planner/multi_agent/adapters.py

Adapter layer that lets the new coordinator runtime reuse the older specialist node logic.

backend/src/agents/travel_planner/multi_agent/coordinator.py

Coordinator decision logic for choosing which specialist or specialist batch runs next.

backend/src/agents/travel_planner/multi_agent/runtime.py

Custom coordinator runtime that supports selective parallel specialist execution and explicit coordination state.

backend/src/agents/travel_planner/multi_agent/schemas.py

Coordination ledger data contracts: roles, tasks, messages, artifacts, and runtime metadata.

backend/src/agents/travel_planner/multi_agent/topology.py

Role topology and delegation policy. It seeds the coordination ledger and defines allowed communication patterns.

backend/src/agents/travel_planner/tooling

Planner tool abstraction layer. These files keep provider-facing capabilities organized as planner-owned tools rather than scattering raw external API logic into every specialist.

backend/src/agents/travel_planner/tooling/__init__.py

Planner tooling module that helps register, validate, or execute planner-owned tools.

backend/src/agents/travel_planner/tooling/airports.py

Planner tooling module that helps register, validate, or execute planner-owned tools.

backend/src/agents/travel_planner/tooling/base.py

Planner tooling module that helps register, validate, or execute planner-owned tools.

backend/src/agents/travel_planner/tooling/contracts.py

Planner tooling module that helps register, validate, or execute planner-owned tools.

backend/src/agents/travel_planner/tooling/factory.py

Planner tooling module that helps register, validate, or execute planner-owned tools.

backend/src/agents/travel_planner/tooling/llm_tools.py

Planner tooling module that helps register, validate, or execute planner-owned tools.

backend/src/agents/travel_planner/tooling/provider_tools.py

Planner tooling module that helps register, validate, or execute planner-owned tools.

backend/src/agents/travel_planner/tooling/registry.py

Planner tooling module that helps register, validate, or execute planner-owned tools.

backend/src/agents/travel_planner/tooling/search_tools.py

Planner tooling module that helps register, validate, or execute planner-owned tools.

backend/src/agents/travel_planner/tooling/travel_tools.py

Planner tooling module that helps register, validate, or execute planner-owned tools.

backend/src/api

Backend HTTP boundary. Modules here accept requests, enforce auth, call services or use cases, and return typed responses.

backend/src/api/main.py

Main route module for the backend API. It exposes health, auth, trip creation, clarification, runtime inspection, admin review, and observability endpoints.

backend/src/application/admin

Application-layer contracts and mappers. These modules shape internal state into API-safe schemas and isolate transport-facing representations from storage and domain models.

backend/src/application/admin/schemas.py

Application-layer schema module. It defines response or transport contracts used outside the core domain layer.

backend/src/application/audit

Application-layer contracts and mappers. These modules shape internal state into API-safe schemas and isolate transport-facing representations from storage and domain models.

backend/src/application/audit/mappers.py

Application-layer mapper module. It converts persistence or domain objects into API-facing schemas.

backend/src/application/audit/schemas.py

Application-layer schema module. It defines response or transport contracts used outside the core domain layer.

backend/src/application/auth

Application-layer contracts and mappers. These modules shape internal state into API-safe schemas and isolate transport-facing representations from storage and domain models.

backend/src/application/auth/schemas.py

Application-layer schema module. It defines response or transport contracts used outside the core domain layer.

backend/src/application/jobs

Application-layer contracts and mappers. These modules shape internal state into API-safe schemas and isolate transport-facing representations from storage and domain models.

backend/src/application/jobs/mappers.py

Application-layer mapper module. It converts persistence or domain objects into API-facing schemas.

backend/src/application/jobs/schemas.py

Application-layer schema module. It defines response or transport contracts used outside the core domain layer.

backend/src/application/observability

Application-layer contracts and mappers. These modules shape internal state into API-safe schemas and isolate transport-facing representations from storage and domain models.

backend/src/application/observability/schemas.py

Application-layer schema module. It defines response or transport contracts used outside the core domain layer.

backend/src/application/trips

Application-layer contracts and mappers. These modules shape internal state into API-safe schemas and isolate transport-facing representations from storage and domain models.

backend/src/application/trips/async_use_cases.py

Application-layer use-case module. It packages one coherent operation behind a narrow execution interface.

backend/src/application/trips/mappers.py

Application-layer mapper module. It converts persistence or domain objects into API-facing schemas.

backend/src/application/trips/use_cases.py

Application-layer use-case module. It packages one coherent operation behind a narrow execution interface.

backend/src/application/workflows

Application-layer contracts and mappers. These modules shape internal state into API-safe schemas and isolate transport-facing representations from storage and domain models.

backend/src/application/workflows/mappers.py

Application-layer mapper module. It converts persistence or domain objects into API-facing schemas.

backend/src/application/workflows/schemas.py

Application-layer schema module. It defines response or transport contracts used outside the core domain layer.

`backend/src/application/workflows/step_mappers.py`

Application-layer mapper module. It converts persistence or domain objects into API-facing schemas.

`backend/src/application/workflows/step_schemas.py`

Application-layer schema module. It defines response or transport contracts used outside the core domain layer.

backend/src/core

Cross-cutting backend infrastructure. Settings, logging, request context, redaction, response shaping, and security live here.

`backend/src/core/config.py`

Typed settings resolution for the backend runtime.

`backend/src/core/logging.py`

Logger configuration helpers.

`backend/src/core/redaction.py`

Helpers for hiding or minimizing sensitive information.

`backend/src/core/request_context.py`

Per-request metadata support such as request ids and contextual logging hooks.

`backend/src/core/response_shaping.py`

Response normalization helpers used before data leaves the backend.

`backend/src/core/security.py`

Actor-context extraction, role checks, and rate-limiting support.

backend/src/db

SQLAlchemy database setup. These modules create the engine and sessions and define the ORM tables.

`backend/src/db/base.py`

Declarative SQLAlchemy base model setup.

`backend/src/db/bootstrap.py`

Database bootstrap utilities.

`backend/src/db/models.py`

SQLAlchemy ORM table definitions for trips, jobs, workflow runs, workflow steps, users, and audit-related records.

`backend/src/db/session.py`

Database engine and session-factory setup.

backend/src/domain/audit

Domain model layer. Business concepts such as trips, jobs, workflows, and audit records are defined here without tying them directly to FastAPI or SQLAlchemy routes.

backend/src/domain/audit/models.py

Domain contract module defining stable business concepts independently of web or database plumbing.

backend/src/domain/audit/repositories.py

Domain contract module defining stable business concepts independently of web or database plumbing.

backend/src/domain/jobs

Domain model layer. Business concepts such as trips, jobs, workflows, and audit records are defined here without tying them directly to FastAPI or SQLAlchemy routes.

backend/src/domain/jobs/models.py

Domain contract module defining stable business concepts independently of web or database plumbing.

backend/src/domain/jobs/repositories.py

Domain contract module defining stable business concepts independently of web or database plumbing.

backend/src/domain/trips

Domain model layer. Business concepts such as trips, jobs, workflows, and audit records are defined here without tying them directly to FastAPI or SQLAlchemy routes.

backend/src/domain/trips/guardrails.py

Domain contract module defining stable business concepts independently of web or database plumbing.

backend/src/domain/trips/models.py

Domain contract module defining stable business concepts independently of web or database plumbing.

backend/src/domain/trips/policies.py

Domain contract module defining stable business concepts independently of web or database plumbing.

backend/src/domain/trips/repositories.py

Domain contract module defining stable business concepts independently of web or database plumbing.

backend/src/domain/trips/services.py

Domain contract module defining stable business concepts independently of web or database plumbing.

backend/src/domain/workflows

Domain model layer. Business concepts such as trips, jobs, workflows, and audit records are defined here without tying them directly to FastAPI or SQLAlchemy routes.

backend/src/domain/workflows/models.py

Domain contract module defining stable business concepts independently of web or database plumbing.

backend/src/domain/workflows/repositories.py

Domain contract module defining stable business concepts independently of web or database plumbing.

backend/src/domain/workflows/step_models.py

Domain contract module defining stable business concepts independently of web or database plumbing.

backend/src/domain/workflows/step_repositories.py

Domain contract module defining stable business concepts independently of web or database plumbing.

backend/src/evals

Evaluation helpers used to compare planner behavior against expected outcomes or golden cases.

backend/src/evals/__init__.py

Evaluation helper module for regressions, scoring, or golden-case execution.

backend/src/evals/golden_cases.py

Evaluation helper module for regressions, scoring, or golden-case execution.

backend/src/evals/regressions.py

Evaluation helper module for regressions, scoring, or golden-case execution.

backend/src/evals/runner.py

Evaluation helper module for regressions, scoring, or golden-case execution.

backend/src/evals/scoring.py

Evaluation helper module for regressions, scoring, or golden-case execution.

backend/src/messaging

Async execution plumbing. These modules connect the application to Redis-backed queue behavior and background jobs.

backend/src/messaging/background_jobs.py

Background-job helpers used to bridge business logic into queued execution.

backend/src/messaging/redis_queue.py

Redis-backed queue builder used by asynchronous workflow execution.

backend/src/persistence/memory

In-memory repository implementations for tests or lightweight flows.

backend/src/persistence/memory/job_repository.py

In-memory repository implementation used where lightweight storage behavior is sufficient.

backend/src/persistence/memory/trip_repository.py

In-memory repository implementation used where lightweight storage behavior is sufficient.

backend/src/persistence/memory/workflow_run_repository.py

In-memory repository implementation used where lightweight storage behavior is sufficient.

backend/src/persistence/memory/workflow_run_step_repository.py

In-memory repository implementation used where lightweight storage behavior is sufficient.

backend/src/persistence/postgres

Postgres-backed repository implementations used by the real backend runtime.

backend/src/persistence/postgres/__init__.py

Postgres repository implementation that persists business and runtime records durably.

backend/src/persistence/postgres/audit_repository.py

Postgres repository implementation that persists business and runtime records durably.

backend/src/persistence/postgres/job_repository.py

Postgres repository implementation that persists business and runtime records durably.

backend/src/persistence/postgres/trip_repository.py

Postgres repository implementation that persists business and runtime records durably.

backend/src/persistence/postgres/user_repository.py

Postgres repository implementation that persists business and runtime records durably.

backend/src/persistence/postgres/workflow_run_repository.py

Postgres repository implementation that persists business and runtime records durably.

backend/src/persistence/postgres/workflow_run_step_repository.py

Postgres repository implementation that persists business and runtime records durably.

backend/src/providers

External provider client layer. These modules know how to talk to OpenAI, Tavily, SerpApi, WeatherAPI, and Aviationstack.

backend/src/providers/__init__.py

Provider client module that wraps an external API behind repository-owned code.

backend/src/providers/base.py

Provider client module that wraps an external API behind repository-owned code.

backend/src/providers/factory.py

Central factory that builds provider clients from configured settings.

backend/src/providers/llm.py

OpenAI client abstractions for model-backed structured generation.

backend/src/providers/search.py

Search-provider clients, including Tavily and SerpApi-style web lookups.

backend/src/providers/travel.py

Travel-provider clients such as weather and flight-enrichment access.

backend/src/services

Service layer. This is where orchestration-aware business operations are coordinated across repositories, runtimes, and policies.

backend/src/services/audit_service.py

Durable audit-event recording and lookup service.

backend/src/services/auth_service.py

Authentication service for registration, login, and user-profile lookups.

backend/src/services/clarification_copilot_service.py

Clarification copilot service that asks focused follow-up questions before heavy workflow execution begins.

backend/src/services/observability_service.py

Run-trace and metrics service used for admin and operator visibility.

backend/src/services/operator_review_service.py

Service for review queues, approval decisions, and operator-facing trip inspection.

backend/src/services/readiness_service.py

Dependency readiness service for startup and health reporting.

backend/src/services/travel_planner_service.py

Main synchronous trip-planning service that bridges trip requests, planner runtime, and persistence.

backend/src/services/workflow_runtime_service.py

Workflow execution service for job creation, run tracking, step tracking, cancellation, rerun, and queue submission.

backend/src/workers

Worker-process entrypoints that run queued workflow jobs outside the web server.

backend/src/workers/workflow_worker.py

Worker-side module that runs queued workflow tasks.

backend/tests

Backend automated tests. These modules describe the intended behavior of configuration, orchestration, providers, observability, security, and resilience.

backend/tests/conftest.py

Automated backend test module for conftest. It captures intended behavior for one risk-prone area of the system.

backend/tests/test_api_security.py

Automated backend test module for test api security. It captures intended behavior for one risk-prone area of the system.

backend/tests/test_application_layers.py

Automated backend test module for test application layers. It captures intended behavior for one risk-prone area of the system.

`backend/tests/test_config.py`

Automated backend test module for test config. It captures intended behavior for one risk-prone area of the system.

`backend/tests/test_destination_research_agent.py`

Automated backend test module for test destination research agent. It captures intended behavior for one risk-prone area of the system.

`backend/tests/test_domain_layers.py`

Automated backend test module for test domain layers. It captures intended behavior for one risk-prone area of the system.

`backend/tests/test_eval_suite.py`

Automated backend test module for test eval suite. It captures intended behavior for one risk-prone area of the system.

`backend/tests/test_multi_agent_coordination.py`

Automated backend test module for test multi agent coordination. It captures intended behavior for one risk-prone area of the system.

`backend/tests/test_observability.py`

Automated backend test module for test observability. It captures intended behavior for one risk-prone area of the system.

`backend/tests/test_operator_review.py`

Automated backend test module for test operator review. It captures intended behavior for one risk-prone area of the system.

`backend/tests/test_provider_clients.py`

Automated backend test module for test provider clients. It captures intended behavior for one risk-prone area of the system.

`backend/tests/test_provider_live_smoke.py`

Automated backend test module for test provider live smoke. It captures intended behavior for one risk-prone area of the system.

`backend/tests/test_reference_links.py`

Automated backend test module for test reference links. It captures intended behavior for one risk-prone area of the system.

`backend/tests/test_runtime_resilience.py`

Automated backend test module for test runtime resilience. It captures intended behavior for one risk-prone area of the system.

`backend/tests/test_tooling_layer.py`

Automated backend test module for test tooling layer. It captures intended behavior for one risk-prone area of the system.

`backend/tests/test_workflow_runtime.py`

Automated backend test module for test workflow runtime. It captures intended behavior for one risk-prone area of the system.

frontend/src

Frontend source root. It contains the application router, global CSS, route-level pages, reusable components, shared layouts, and the client-side planner integration layer.

`frontend/src/App.tsx`

Top-level React router and page composition module.

`frontend/src/index.css`

Global frontend styles and visual tokens.

`frontend/src/index.tsx`

React browser entrypoint that mounts the app.

`frontend/src/react-app-env.d.ts`

TypeScript environment declaration module for the React toolchain.

frontend/src/app

Route-level React pages. Each module corresponds to a major user-facing screen such as new trip, clarification, itinerary, budget, review, or research.

`frontend/src/app/AppPage.module.css`

Page-specific CSS module controlling one route's layout and visual treatment.

frontend/src/app/budget

Route-level React pages. Each module corresponds to a major user-facing screen such as new trip, clarification, itinerary, budget, review, or research.

`frontend/src/app/budget/Budget.tsx`

Budget artifact screen for spend analysis and optimization notes.

frontend/src/app/clarification

Route-level React pages. Each module corresponds to a major user-facing screen such as new trip, clarification, itinerary, budget, review, or research.

`frontend/src/app/clarification/Clarification.tsx`

Clarification and workflow-launch screen that drives the guided copilot and shows runtime progress.

frontend/src/app/dashboard

Route-level React pages. Each module corresponds to a major user-facing screen such as new trip, clarification, itinerary, budget, review, or research.

`frontend/src/app/dashboard/Dashboard.module.css`

Page-specific CSS module controlling one route's layout and visual treatment.

`frontend/src/app/dashboard/Dashboard.tsx`

Landing dashboard for the frontend experience.

frontend/src/app/itinerary

Route-level React pages. Each module corresponds to a major user-facing screen such as new trip, clarification, itinerary, budget, review, or research.

`frontend/src/app/itinerary/Itinerary.tsx`

Itinerary artifact screen for day-by-day trip plans.

frontend/src/app/login

Authentication screens. These files implement user login and signup flows in the browser.

`frontend/src/app/login/Login.module.css`

Page-specific CSS module controlling one route's layout and visual treatment.

`frontend/src/app/login/Login.tsx`

Browser login screen.

`frontend/src/app/login/Signup.tsx`

Browser signup screen.

frontend/src/app/new-trip

Route-level React pages. Each module corresponds to a major user-facing screen such as new trip, clarification, itinerary, budget, review, or research.

`frontend/src/app/new-trip/NewTrip.tsx`

Trip intake screen where the user enters the base request.

frontend/src/app/research

Route-level React pages. Each module corresponds to a major user-facing screen such as new trip, clarification, itinerary, budget, review, or research.

`frontend/src/app/research/Research.tsx`

Research artifact screen for destination context and evidence.

frontend/src/app/review

Route-level React pages. Each module corresponds to a major user-facing screen such as new trip, clarification, itinerary, budget, review, or research.

`frontend/src/app/review/Review.tsx`

Review and governance artifact screen.

frontend/src/components

Reusable UI building blocks shared across multiple pages.

`frontend/src/components/AccountBar.module.css`

Component-scoped CSS module for a reusable UI building block.

`frontend/src/components/AccountBar.tsx`

Reusable React component shared across one or more screens.

`frontend/src/components/AccountCard.module.css`

Component-scoped CSS module for a reusable UI building block.

`frontend/src/components/AccountCard.tsx`

Reusable React component shared across one or more screens.

`frontend/src/components/Card.module.css`

Component-scoped CSS module for a reusable UI building block.

`frontend/src/components/Card.tsx`

Reusable React component shared across one or more screens.

`frontend/src/components/Chat.module.css`

Component-scoped CSS module for a reusable UI building block.

`frontend/src/components/Chat.tsx`

Reusable React component shared across one or more screens.

`frontend/src/components/Checklist.module.css`

Component-scoped CSS module for a reusable UI building block.

`frontend/src/components/Checklist.tsx`

Reusable React component shared across one or more screens.

`frontend/src/components/ExportPanel.module.css`

Component-scoped CSS module for a reusable UI building block.

`frontend/src/components/ExportPanel.tsx`

Reusable React component shared across one or more screens.

`frontend/src/components/FilterBar.module.css`

Component-scoped CSS module for a reusable UI building block.

`frontend/src/components/FilterBar.tsx`

Reusable React component shared across one or more screens.

`frontend/src/components/Header.module.css`

Component-scoped CSS module for a reusable UI building block.

`frontend/src/components/Header.tsx`

Reusable React component shared across one or more screens.

`frontend/src/components/ItineraryDayCard.module.css`

Component-scoped CSS module for a reusable UI building block.

`frontend/src/components/ItineraryDayCard.tsx`

Reusable React component shared across one or more screens.

`frontend/src/components/Sidebar.module.css`

Component-scoped CSS module for a reusable UI building block.

`frontend/src/components/Sidebar.tsx`

Reusable React component shared across one or more screens.

`frontend/src/components/StatusBadge.module.css`

Component-scoped CSS module for a reusable UI building block.

`frontend/src/components/StatusBadge.tsx`

Reusable React component shared across one or more screens.

`frontend/src/components/TravelRibbon.module.css`

Component-scoped CSS module for a reusable UI building block.

`frontend/src/components/TravelRibbon.tsx`

Reusable React component shared across one or more screens.

`frontend/src/components/UtilityPanel.module.css`

Component-scoped CSS module for a reusable UI building block.

`frontend/src/components/UtilityPanel.tsx`

Reusable React component shared across one or more screens.

frontend/src/layouts

Shared application layout modules that provide page chrome and structural consistency.

`frontend/src/layouts/AppLayout.module.css`

Layout module for shared frontend structure.

`frontend/src/layouts/AppLayout.tsx`

Shared layout and page-chrome wrapper for the frontend.

frontend/src/lib

Frontend integration library. This is where API calls, local storage helpers, and shared frontend types live.

`frontend/src/lib/planner.test.ts`

Frontend tests for the planner client and storage helpers.

`frontend/src/lib/planner.ts`

Frontend planner client layer. It centralizes HTTP calls, local persistence, runtime polling helpers, and shared UI-facing types.

Reading Sequence Recommendation

- Start with `backend/src/main.py`, `backend/src/bootstrap.py`, and `backend/src/api/main.py` to understand process assembly and HTTP surface area.
- Move to `backend/src/agents/travel_planner/schemas.py`, `state.py`, `graph.py`, and `multi_agent/runtime.py` to understand planner contracts and orchestration.
- Then read `backend/src/services/workflow_runtime_service.py` and the persistence repositories to understand how async runs become durable records.
- After that, read `frontend/src/lib/planner.ts`, `frontend/src/app/new-trip/NewTrip.tsx`, and `frontend/src/app/clarification/Clarification.tsx` to understand the browser-side integration flow.
- Finally, read the backend tests alongside the corresponding modules to see what the code promises to preserve.

Closing Note

This manual is meant to reduce search cost. A strong engineering manual does not replace reading the code; it makes reading the code more efficient by telling you where each responsibility lives before you dive into implementation detail.